Glovo

Predicción de ETA en Plataforma Glovo

Análisis predictivo basado en 63.000+ pedidos reales de París

MAE Final

6.4 min

LightGBM + Saturation
Modelo final seleccionado

Reto de Inteligencia Artificial · Master en Data Analytics for Business · BSM · Mayo 2026

El Problema de Negocio

🛵 Glovo App

🏠 Wall
ETA visible antes de entrar

🏪 Store
ETA en menú del restaurante

🛒 Checkout
ETA con info del carrito

⏱️ Countdown
Cuenta regresiva post-pedido

📈 Conversión
ETAs bajos → mayor probabilidad de compra

⚖️ Expectativas
ETA incumplido → frustración
y churn del cliente

💰 Rentabilidad
Pedido tardío → compensación
→ mayor CPO

⚠️ Asimetría de costes clave

Prometer 30 min y entregar en 20 → satisfacción o incongruencia logístico
Prometer 30 min y entregar en 45 → queja, refund, churn

Esta asimetría orienta nuestra elección de métricas y modelo.

Explicado por: Juan David Manrique

El Dataset

63.646
pedidos reales

21
variables originales

3
meses de datos
(París, oct–dic 2019)

<1%
filas eliminadas
tras limpieza

Bloques de variables

🕐 Temporales
activation_time
courier_started
pickup_time
delivery_time

📍 Geográficas
pickup_lat/lon
delivery_lat/lon
→ distance_km derivada

🏪 Restaurante
store_name
store_address_id
vertical

⚙️ Operacional
saturation
tranport_type
courier_id

💶 Económicas
gtv
delivery_fee
description

🎯 Variable target: total_time = delivery_time − activation_time (minutos percibidos por el cliente)

Explicado por: Juan David Manrique

Metodología: 5 Fases

01

Auditoría de datos

¿Qué tenemos?
¿Es fiable?

02

EDA

¿Qué patrones hay?
¿Qué variables importan?

03

Ingeniería de variables

¿Cómo expresamos mejor la información?

04

Modelado

¿Qué complejidad necesitamos?

05

Interpretación

¿Qué nos dice el modelo?

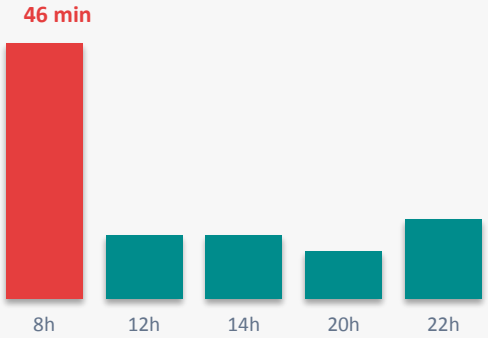
Decisiones clave: ✓ Split temporal (no aleatorio) para respetar causalidad · ✓ MAE como métrica principal (distribución asimétrica) · ✓ Complejidad creciente justificada por evidencia

Explicado por: Juan David Manrique

EDA: Lo que los datos realmente dicen

Tres hallazgos que contradicen la intuición habitual sobre delivery

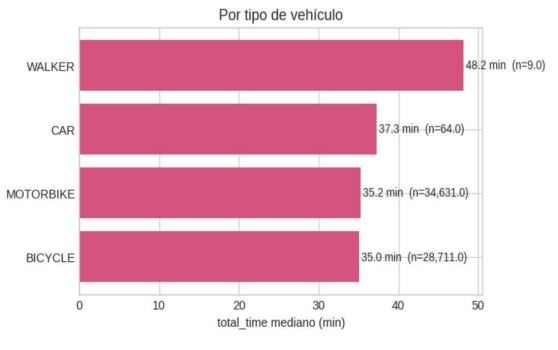
🕐 Pico real a las 8h



Lunch y dinner son los momentos MÁS RÁPIDOS (33-34 min). El problema está en la mañana: poca demanda pero poquísima oferta de couriers.

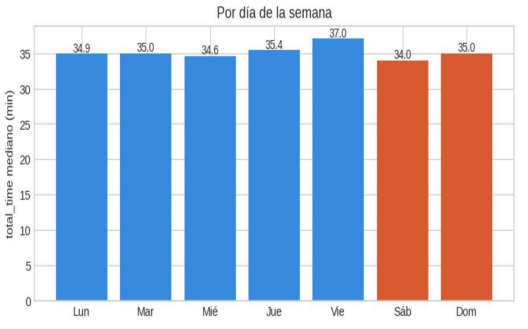
Implicación: reforzar flota 8-11h, no lunch/dinner.

🚲 Bici ≈ Moto en París



BICYCLE: 35.0 min ≈ MOTORBIKE: 35.2 min
En París, carriles dedicados y atajos compensan la menor velocidad punta de las bicis.
El tipo de vehículo tiene impacto muy limitado.

📅 Viernes ≠ Fin de semana



Más lento: Viernes (37.0 min)
Más rápido: Sábado (34.0 min)
La intuición 'fines de semana = lentos' no se sostiene en estos datos.

Implicación metodológica: no construimos features basadas en la intuición — las derivamos de lo que muestran los datos.

Explicado por: Ana Cris - Juan David

El Caso Saturation: ¿Data Leakage o Variable Legítima?

⚠️ Correlación anómala detectada

saturation ↔ total_time: **ρ = 0.778**
La siguiente más alta (distance_km): ρ = 0.353
Una sola variable explicando el **61% de la varianza** → lo cual necesitaba mayor revisión.

✓ Veredicto: Variable legítima

Captura el ratio carga/capacidad de la flota.
No está contaminada con información del target.
Su inclusión aporta 2.44 min de mejora real en MAE.

total_time = prep_time + travel_time

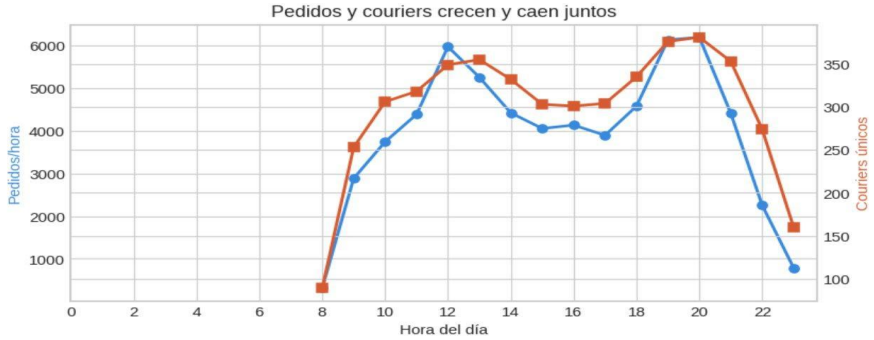
Las siguientes 3 diapositivas detallan cada test con sus gráficas.

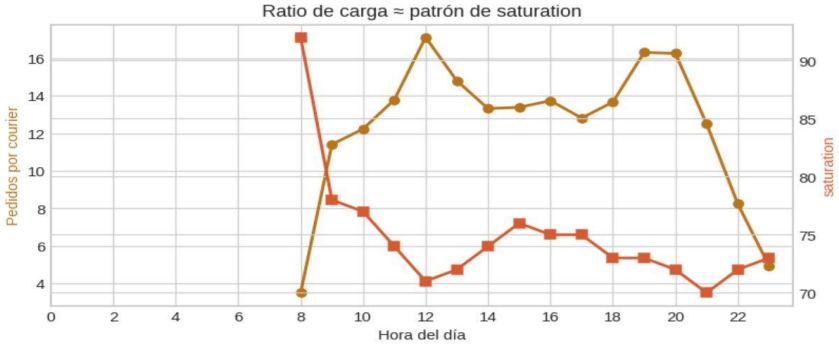
Explicado por: Ana Cristina Castro

TEST 1, 2/ 3

Test Heurístico: ¿Sigue Saturation el ciclo de demanda?

Hipótesis: si saturation mide presión real del sistema, debería tener picos en las horas de mayor demanda (lunch ~13h, dinner ~20h), igual que total_time.





Resultado: la premisa era falsa. Saturation plana en horas pico NO es evidencia de leakage — es evidencia de que la flota se escala correctamente. En lunch/dinner hay 4× más couriers que a las 8h.

Explicado por: Ana Cristina Castro

TEST 2 / 3

Within-Hour Correlation: ¿Aporta Saturation Información Independiente?

Hipótesis: si saturation fuera leakage que entra por la hora del día, al fijar la hora la correlación con total_time debería desaparecer (caer a ~0).

Correlación dentro de cada hora del día

h=8	0.853
h=10	0.806
h=12	0.772
h=13	0.779
h=14	0.791
h=17	0.764
h=19	0.750
h=20	0.757
h=21	0.769
h=23	0.732

Rango: 0.732 – 0.853
Comparar con correlación bruta: 0.778

Resultado del test

Correlación bruta

0.778

Within-hour (promedio)

0.773

↕ **diferencia: 0.005**

✓ Si fuera leakage, los números colapsarían a ~0 al fijar la hora. Se mantienen en 0.73–0.85. Esto es información residual real.

Explicado por: Yana Alyeksyeyeva

TEST 3 / 3

Residualización: ¿Viene la Correlación de la Hora del Día?

Hipótesis: si la correlación saturation↔total_time viniera de la hora del día, al restar a cada variable el promedio de su hora, la correlación entre residuos debería caer drásticamente.

¿Qué hace la residualización?

1. Calculamos el promedio de saturation por hora del día
2. Restamos ese promedio a cada pedido → sat_resid
3. Hacemos lo mismo con total_time → tt_resid
4. Calculamos corr(sat_resid, tt_resid)
5. Si la hora explicaba la relación, esta correlación debería ser ≈ 0

El resultado

Correlación BRUTA		Correlación RESIDUAL
0.778	→	0.777

Diferencia: 0.0015 → prácticamente cero

La hora del día explica esencialmente NADA de la asociación entre saturation y total_time. Si fuera leakage horario, habríamos visto una caída de varios puntos, no de 0.001.

✓ Los 3 tests descartan leakage. Decisión: incluir saturation en el modelo final. Su exclusión habría costado 2.44 min de MAE injustificadamente.

Explicado por: Yana Alyeksyeyeva

Ingeniería de Variables

Solo añadimos variables con justificación analítica explícita

📍 distance_km

Haversine entre pickup y delivery

Predictor físico del tiempo de viaje. Corr = 0.59 con travel_time.

🗼 dist_to_center
inside_periph

Distancia al centro de París
Pickup dentro del Périphérique (<4km)

Zona urbana afecta velocidad. 7.48 min/km dentro vs 7.09 fuera (+6%).

🕐 hour / dow
meal_period

Componentes temporales
Desagregación por período del día

Pico real a las 8h. Meal_period discretiza sin asumir 'rush = lento'.

🏷️ pre_assigned
dispatch_delay_pos

Courier asignado antes del pedido
Tiempo hasta asignación (clipped ≥0)

Pre-asignación ⟶ 33.1 min vs 39.9 min sin asignar. Efecto de ~7 min.

📦 item_count

Conteo de ítems con regex sobre description

Corr débil (0.11 con prep_time). Se mantiene por posibles interacciones.

⚡ saturation

Ratio carga/capacidad de la flota

Validada en 3 tests. Variable más predictiva del modelo final (17%).

Variables descartadas: rating (57% nulos) · bad_rating_reason (96% nulos) · is_rush (hipótesis invalidada por EDA)

Explicado por: Yana Alyeksyeyeva

Comparativa de Modelos: Complejidad Creciente

#	Modelo	MAE (min)	RMSE	R^2	Mejora vs baseline
1	Baseline (mediana)	12.28	17.21	-0.10	—
2	Regresión Ridge	9.16	12.59	0.41	-3.12 min
3	Random Forest	9.15	12.55	0.42	-3.13 min
4	LightGBM básico	9.02	12.36	0.43	-3.26 min
5	LightGBM + categóricas	8.87	12.21	0.45	-3.41 min
6	**LightGBM + cats + saturation**	**6.43**	**8.21**	**0.75**	**-5.85 min**

Baseline → Ridge

−3.1 min
(mayor salto después de saturation)

RF ≈ Ridge

Complejidad ≠ mejora automática

+ saturation

−2.44 min
(el que justifica toda la validación)

Explicado por: Yana - Juan David

Demo: Predicción para un Caso Concreto

Así se usa el modelo en producción

Demo: predicción para un caso concreto

Mostramos cómo se usa el modelo en producción con un pedido de ejemplo.

```python
ejemplo = pd.DataFrame([{
    'distance_km':          2.1,
    'dist_to_center':       0.8,
    'inside_periph':        1,
    'hour':                 20,
    'dow':                  4,
    'item_count':           3,
    'gtv':                  22.50,
    'pre_assigned':         1,
    'dispatch_delay_pos':   0,
    'vertical_cat':         0,
    'tranport_type_cat':    0,
    'meal_period_cat':      0,
    'store_name_cat':       100,
    'saturation':           75,
}])

pred = model_final_full.predict(ejemplo[FEATURES_FINAL])[0]

print('Pedido de ejemplo:')
print('  • Viernes 20h (hora dinner — históricamente rápida)')
print('  • Restaurante en el centro de París (dentro del Périph)')
print('  • 2,1 km de distancia al cliente')
print('  • 3 ítems, 22,50€')
print('  • Courier en moto, pre-asignado')
print('  • saturation = 75 (carga moderada-alta del sistema)')
print(f'\n→ ETA predicho: {pred:.1f} minutos')
print(f'→ Display al usuario: "Tu pedido llegará en aproximadamente {round(pred):.0f} minutos"')
```

```
Pedido de ejemplo:
  • Viernes 20h (hora dinner — históricamente rápida)
  • Restaurante en el centro de París (dentro del Périph)
  • 2,1 km de distancia al cliente
  • 3 ítems, 22,50€
  • Courier en moto, pre-asignado
  • saturation = 75 (carga moderada-alta del sistema)

→ ETA predicho: 29.8 minutos
→ Display al usuario: "Tu pedido llegará en aproximadamente 30 minutos"
```

Pedido de ejemplo

Viernes 20h
hora dinner — históricamente rápida

Centro de París
dentro del Périphérique

2,1 km
distancia al cliente

3 ítems · 22,50€
valor del pedido

Moto · pre-asignado
courier ya en ruta

Saturation = 75
carga moderada-alta del sistema

ETA predicho

≈ 30 minutos

El modelo predice 29,8 min en tiempo real, a partir de las variables disponibles en el momento de la activación del pedido.

Explicado por: Yana Alyeksyeyeva

Conclusiones

🎯 Modelo robusto

MAE 6,4 min · R^2 0,75 · validado temporalmente

🔬 Hallazgo metodológico

Saturation (ρ=0,78) pasó 3 tests rigurosos.
Excluirla habría costado 2,44 min MAE.

💡 Insight operacional

El pico real está a las 8h, no en lunch/dinner.
Ahí está el margen de mejora.

Limitaciones del trabajo

📅 Solo 3 meses de datos

No captura estacionalidad anual.
Navidad, verano y festivos quedan fuera.

🗺️ Solo París

Geografía radial y específica.
No generalizable a Barcelona.

📐 Distancia en línea recta

Haversine vs ruta real por carretera.
OSRM mejoraría travel_time.

🚴 Sin datos del courier

Experiencia, historial
individual no están disponibles.

🌧️ Sin datos externos

Sin meteorología, eventos urbanos,
huelgas no casos de tráfico extraordinarios.

Explicado por: Ana Cristina Castro